

Experiment 5

Student Name: Prateek Verma

UID: 25MCC20062

Branch: MCA (CC & DEVOPS)

Section/Group: 25MCC-101/A

Semester: IInd

Date of Performance: 17/02/2026

Subject Name: Technical Training

Subject Code: 25CAP-652

Title: Creating and using cursors for row-by-row processing in a database.

1. Aim:

- a. To gain hands-on experience in creating and using cursors for row-by-row processing in a database, enabling sequential access and manipulation of query results for complex business logic.

2. Software Requirements:

- a. PostgreSQL (Database Server)
- b. pgAdmin
- c. OS: Windows

3. Objective:

- a. Sequential Data Access: To understand how to fetch rows one by one from a result set using cursor mechanisms.
- b. Row-Level Manipulation: To perform specific operations or calculations on individual records that require conditional procedural logic.
- c. Resource Management: To learn the lifecycle of a cursor: Declaring, Opening, Fetching, and importantly, Closing and Deallocating to manage system memory.
- d. Exception Handling: To handle cursor-related errors and performance considerations during large-scale data iteration.

4. Theory: While SQL is generally set-oriented, certain tasks require a procedural approach where we process one row at a time. This is where Cursors are used:

- a. Cursor Types: Cursors can be Implicit (managed by the system) or Explicit (defined by the developer). They can also be Forward-Only (moving only toward the end) or Scrollable (moving back and forth).
- b. The Lifecycle:
 - DECLARE: Defines the SQL query for the cursor.
 - OPEN: Executes the query and establishes the result set.
 - FETCH: Retrieves a specific row into variables for processing.
 - CLOSE: Releases the current result set.

- DEALLOCATE: Removes the cursor definition from memory.
- c. Use Case: Cursors are ideal for generating row-specific reports, updating balances based on complex historical data, or migrating data where each record needs individual validation.

5. Procedure of the Practical:

- a. Start the system.
- b. Open pgAdmin.
- c. Create and select the database in which you want to perform the experiment.
- d. Establish connection to the database using Alt+Shift+Q.
- e. Run the following queries given in Experiment Steps (6).

6. Experiment Steps:

Base Table:

	emp_id [PK] integer	emp_name character varying (100)	experience_years integer	performance_rating numeric (3,2)	salary numeric (10,2)
1	1	Anay	2	4.50	20000.00
2	2	Bhavna	10	3.00	50000.00
3	3	Chaitanya	5	4.00	35000.00
4	4	Deepak	0	3.50	25000.00
5	5	Esha	8	4.80	60000.00

- a. Implementing a simple forward only cursor.

```
DO $$
```

```
DECLARE
```

```
    emp_cursor CURSOR FOR SELECT emp_id, emp_name, salary FROM Employee;
    emp_record RECORD;
```

```
BEGIN
```

```
    FOR emp_record IN emp_cursor LOOP
```

```
        RAISE NOTICE 'Employee ID: %, Name: %, Salary: %',
            emp_record.emp_id,
            emp_record.emp_name,
            emp_record.salary;
```

```
    END LOOP;
```

```
END $$;
```

```
NOTICE: Employee ID: 1, Name: Anay, Salary: 20000.00
NOTICE: Employee ID: 2, Name: Bhavna, Salary: 50000.00
NOTICE: Employee ID: 3, Name: Chaitanya, Salary: 35000.00
NOTICE: Employee ID: 4, Name: Deepak, Salary: 25000.00
NOTICE: Employee ID: 5, Name: Esha, Salary: 60000.00
DO
```

- b. Complex row-by-row manipulation using cursor to update salaries based on dynamic “experience to performance” ratio.

DO \$\$

DECLARE

emp_cursor CURSOR FOR SELECT emp_id, emp_name, experience_years,
performance_rating, salary FROM Employee;

emp_record RECORD;

v_ratio NUMERIC;

v_bonus_multiplier NUMERIC;

BEGIN

FOR emp_record IN emp_cursor LOOP

v_ratio := emp_record.performance_rating / (emp_record.experience_years + 1);

IF v_ratio >= 1.5 THEN

v_bonus_multiplier := 1.20;

ELSIF v_ratio >= 0.8 THEN

v_bonus_multiplier := 1.10;

ELSE

v_bonus_multiplier := 1.05;

END IF;

UPDATE Employee

SET salary = salary * v_bonus_multiplier

WHERE emp_id = emp_record.emp_id;

RAISE NOTICE 'Employee: % salary incremented by multiplier of %',
emp_record.emp_name, v_bonus_multiplier;

END LOOP;

END \$\$;

```
NOTICE: Employee: Anay salary incremented by multiplier of 1.20
NOTICE: Employee: Bhavna salary incremented by multiplier of 1.05
NOTICE: Employee: Chaitanya salary incremented by multiplier of 1.05
NOTICE: Employee: Deepak salary incremented by multiplier of 1.20
NOTICE: Employee: Esha salary incremented by multiplier of 1.05
DO
```

	emp_id [PK] integer	emp_name character varying (100)	experience_years integer	performance_rating numeric (3,2)	salary numeric (10,2)
1	1	Anay	2	4.50	24000.00
2	2	Bhavna	10	3.00	52500.00
3	3	Chaitanya	5	4.00	36750.00
4	4	Deepak	0	3.50	30000.00
5	5	Esha	8	4.80	63000.00

- c. Exception and status handling ensuring the cursor handles empty result sets and termination signals gracefully.

```
DO $$
DECLARE
    emp_cursor CURSOR FOR SELECT * FROM Employee;
    emp_record RECORD;
    v_ratio NUMERIC;
BEGIN
    OPEN emp_cursor;

    LOOP
        FETCH emp_cursor INTO emp_record;
        EXIT WHEN NOT FOUND;

        BEGIN
            IF emp_record.experience_years < 0 THEN
                RAISE EXCEPTION 'Invalid experience for employee %', emp_record.emp_name;
            END IF;

            v_ratio := emp_record.performance_rating / (emp_record.experience_years + 1);

            UPDATE Employee
            SET salary = salary * (1 + (v_ratio / 10)) -- Dynamic 10% scaled raise
            WHERE emp_id = emp_record.emp_id;

            RAISE NOTICE 'Success: Updated %', emp_record.emp_name;

        EXCEPTION
            WHEN OTHERS THEN
                RAISE WARNING 'Skipping Employee % due to error: %', emp_record.emp_name,
SQLERRM;
        END;

    END LOOP;

    CLOSE emp_cursor;

    EXCEPTION
        WHEN OTHERS THEN
            RAISE EXCEPTION 'Critical Failure in Cursor Processing: %', SQLERRM;
    END $$;
```

```
NOTICE: Success: Updated Anay
NOTICE: Success: Updated Bhavna
NOTICE: Success: Updated Chaitanya
NOTICE: Success: Updated Deepak
NOTICE: Success: Updated Esha
WARNING: Skipping Employee Failing_Fred due to error: Invalid experience for employee Failing_Fred
NOTICE: Success: Updated Null_Nancy
DO
```

Before:

	emp_id [PK] integer	emp_name character varying (100)	experience_years integer	performance_rating numeric (3,2)	salary numeric (10,2)
1	1	Anay	2	4.50	24000.00
2	2	Bhavna	10	3.00	52500.00
3	3	Chaitanya	5	4.00	36750.00
4	4	Deepak	0	3.50	30000.00
5	5	Esha	8	4.80	63000.00
6	6	Failing_Fred	-5	4.00	30000.00
7	7	Null_Nancy	3	[null]	40000.00

After:

	emp_id [PK] integer	emp_name character varying (100)	experience_years integer	performance_rating numeric (3,2)	salary numeric (10,2)
1	1	Anay	2	4.50	27600.00
2	2	Bhavna	10	3.00	53931.82
3	3	Chaitanya	5	4.00	39200.00
4	4	Deepak	0	3.50	40500.00
5	5	Esha	8	4.80	66360.00
6	6	Failing_Fred	-5	4.00	30000.00
7	7	Null_Nancy	3	[null]	[null]

7. Input/Output Details:

- The input for the experiment is the SQL queries mentioned in Experiment Steps (5).
- The output for each query is attached after the query itself.

8. Learning Outcome

- Cursor Implementation: Ability to design, implement, and manage cursors to solve row-wise processing problems.
- Lifecycle Mastery: Gaining knowledge of the correct syntax for declaring, opening, fetching, and closing cursors.
- Error Prevention: Understanding how to properly handle row-by-row processing exceptions and prevent memory leaks via deallocation.
- Analytical Thinking: Applying cursor-based logic to solve real-world scenarios like multi-level payroll adjustments or data migrations.